

Laboratório de Concorrência · Aula 10

Rodrigo Augusto da Silva Virgilino · RA 24139528-2 · Turma A

Gerado em 09/10/2026, 20:36:17 · tempo registrado no cronômetro: 66:38

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando uma tarefa precisa esperar outra concluir uma atividade para poder continuar. Um exemplo seria: em uma construção o pintor precisa esperar o terminar a obra para começar a pintar.

A sincronização de competição ocorre quando duas ou mais tarefas precisam utilizar o mesmo recurso. Exemplo: duas pessoas querem usar a mesma impressora ao mesmo tempo, mas cada uma precisa esperar seu tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

1017991

2ª execução (n = 1.000.000)

1052053

3ª execução (n = 1.000.000)

1051771

Saída do programa

não respondido

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

O valor esperado é 2 milhões (cada thread incrementa 1 milhão), porém, cada thread acessa e modifica a mesma variável causando condição de corrida, isso resulta em valores inesperados.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

a thread 1 lê o valor; a thread 2 também lê o valor; a thread 1 soma e grava; a thread 2 soma ao valor e também grava.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Resulta em 2000, pois existe menos chances de ocorrer uma condição de corrida. Isso é perigoso porque o programa pode executar normalmente durante os testes, mas em produção pode apresentar erros.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
BBBAAA → 7
ABABAB → 6
AABBAB → 6
BBABAA → 4
AABABB → 6
BBAABA → 4
BBAAAB → 6
BABABA → 4
ABBABA → 4

Saída do programa

não respondido

```
Console I/O AI Agent
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

20 ordens, 4 resultados diferentes. Apenas uma ordem é considerado correto, seria: AAABBB

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

BABABA:

B lê 3;

A lê 3;

B calculou $3 \times 2 = 6$;

A calculou $3 + 1 = 4$;

B grava 6;

A grava 4, sobrescrevendo 6.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que operação atômica por não precisa de travas como nas outras

Saída do programa

não respondido

```
Console I/O AI Agent
esperado:      2000000
sem sincronizacao: 1426468 em 2 ms
synchronized:  2000000 em 181 ms
AtomicInteger:  2000000 em 22 ms
```

```
Console I/O AI Agent
esperado:      2000000
sem sincronizacao: 1957986 em 5 ms
synchronized:  2000000 em 106 ms
AtomicInteger:  2000000 em 72 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As duas deram o valor correto. A versão correta costuma ser mais lenta devido a condição de corrida.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não, pois ele garante apenas que cada saldo seja atualizado individualmente, não protege a transferência inteira. Seria necessário usar sincronização, como o synchronized ou travas.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, ocorrerá deadlock. A thread A bloqueia o recurso X e espera Y, enquanto a thread B bloqueia Y e espera X.

Saídas (antes e depois da correção)

não respondido

```
Console I/O AI Agent
IMPASSE: 2 threads presas
```

```
Console I/O AI Agent
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque a thread A bloqueou o recurso X e ficou esperando Y, enquanto a thread B bloqueou o recurso Y e ficou esperando X. Como nenhuma das duas liberava o recurso que possuía, ambas ficaram esperando uma pela outra, causando um deadlock

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pega os recursos sempre na mesma ordem evita o deadlock porque impede que as threads fiquem esperando uma pela outra. Assim, uma thread utiliza os recursos enquanto a outra aguarda liberar.

Ex: Em um sistema de estoque, duas operações podem bloquear produtos diferentes e ficar esperando uma pela outra. Acessar os produtos sempre na mesma ordem evita deadlock.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

3 trabalhadores: 200ms;

1 trabalhador: 600ms;

6 trabalhados: 100ms

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido

```
Console I/O AI Agent  
trabalhador 1 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 1 fez a tarefa 3  
trabalhador 1 fez a tarefa 4  
trabalhador 1 fez a tarefa 5  
trabalhador 1 fez a tarefa 6  
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
Console I/O AI Agent  
trabalhador 2 fez a tarefa 3  
trabalhador 1 fez a tarefa 2  
trabalhador 3 fez a tarefa 1  
trabalhador 3 fez a tarefa 6  
trabalhador 2 fez a tarefa 4  
trabalhador 1 fez a tarefa 5  
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
Console I/O AI Agent  
trabalhador 5 fez a tarefa 6  
trabalhador 6 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 3 fez a tarefa 4  
trabalhador 4 fez a tarefa 5  
trabalhador 2 fez a tarefa 3  
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, o programa leva cerca de 600 ms, pois executa as 6 tarefas sozinho. Com 3 trabalhadores, leva 200 ms, já que cada um realiza 2 tarefas. Com 6 trabalhadores, leva 100 ms, pois todas podem ser executadas simultaneamente.

O programa não precisa de synchronized nem de travas porque utiliza canais para trocar mensagens, sem que os trabalhadores precisem modificar uma variável compartilhada. Cada tarefa é recebida por apenas um trabalhador, evitando condições de corrida.

Saídas (antes e depois da troca)

não respondido

```
Console I/O AI Agent Exit 1
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2
```

```
Console I/O AI Agent
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O programa trava porque o canal não tem buffer, então o envio fica esperando alguém receber a mensagem. Como o recebimento está na linha seguinte, ele nunca é executado, causando um deadlock.

Com um canal de capacidade 1, a mensagem pode ser armazenada sem precisar esperar um receptor. Assim, o programa continua e consegue receber o valor normalmente.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O programa não vai compilar, pois o Rust não permite que uma thread altere uma variável de outra sem garantir a segurança do acesso.

Saídas (o erro e o resultado com move)

Primeira saída:

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned by the current function
```

Segunda saída:

```
0
```

Saída do programa

não respondido

```
Console I/O AI Agent
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Imprime 0 porque o move faz a thread receber uma cópia de contador. Assim, a thread incrementa sua própria cópia para 1, enquanto a variável original em main continua com o valor 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro foi descoberto durante a execução, quando o contador apresentou valores incorretos. Já em Rust, o compilador identificou o problema antes de executar o programa.

A vantagem do Rust é evitar erros de concorrência antes que aconteçam. O custo é que o programador precisa seguir regras mais rígidas e utilizar mecanismos como Arc e Mutex para compartilhar dados com segurança.

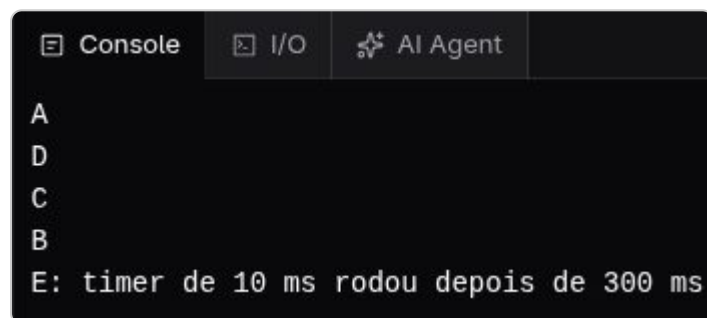
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, B

Saída do programa

não respondido



```
Console I/O AI Agent
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem foi A, B, C e B. O B saiu depois porque o setTimeout, mesmo com tempo 0, precisa esperar o código síncrono terminar. Por isso, D aparece primeiro. Já o C aparece antes de B porque as microtarefas das Promises tem prioridade sobre as tarefas do setTimeout.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer atrasou porque o laço ocupado(300) bloqueou a única thread do JavaScript por 300 ms, impedindo a execução de outras tarefas. Em um servidor Node.js, isso significa que um cálculo pesado pode atrasar as respostas para vários usuários, deixando o servidor lento ou até sem responder temporariamente.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo.

Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida, em que as duas pessoas comprem o mesmo ingresso porque o sistema verifica a disponibilidade ao mesmo tempo. Para evitar isso, usaria sincronização, garantindo que apenas uma compra por vez possa verificar e atualizar a quantidade de ingressos disponíveis.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Que o programa pode funcionar várias vezes e mesmo assim apresentar erros inesperados por causa das condições de corrida.